Don't Read the Whole Guideline: Decision Orchestration for Complex Annotation with Small Models and a Handful of Examples

Anonymous ACL submission

Abstract

Large language models (LLMs) can help bootstrap annotation when gold data is scarce using guidelines with few examples, but they often struggle with specialized annotation conventions and formats, while prompting alone is not reliable and interpretable on how to systematically revise instructions to correct their errors. We propose *decision orchestration*, a framework that turns annotation guidelines into an executable procedure in which code controls structure and models make local decisions. A reflection module serves as a moderation process, using decision traces and small batches of gold annotations to diagnose errors and iteratively refine the procedure. On UMR and PARSEME in two languages, reflection with just a few annotated sentences using small language models or decision models outperforms larger models, suggesting that decision orchestration can serve as a natural first step for future annotation projects, which is to turn the guideline into an executable procedure, then refine it through fine-grained evaluation and a few examples.

1 Introduction

Annotating a complex annotation scheme takes two things: an understanding of the structure the scheme describes, and the scheme's own codebook, the conventions that make one annotation right and a plausible alternative wrong (Kim et al., 2026). Early in a project, a new scheme, language or domain has a guideline but little gold. With enough gold, supervised models learn both (Bevilacqua et al., 2021; Bai et al., 2022); with a handful of sentences they learn neither. Large language models bring the first ingredient for free: they understand sentences, events and referents well enough to bootstrap an annotation. What remains is to give them the codebook.

For simple labeling, the guideline fits into the prompt (Gilardi et al., 2023; Tan et al., 2024). Complex schemes pose two challenges: a) *guideline following*: the UMR guidelines exceed 300 pages (Van Gysel et al., 2021), burying relevant rules in long contexts (Liu et al., 2024) and an annotation must satisfy dozens of them at once, where following degrades with each added instruction (Harada et al., 2025). b)*interpretable moderation*: correcting annotation errors requires diagnosing and refining how guidelines are applied (Kim et al., 2026). When one prompt produces the whole annotation, errors are difficult to trace to specific decisions, leaving prompt revisions poorly targeted.

We propose **decision orchestration** (Figure 1), which compiles annotation guidelines into executable steps. Code enforces the output format, annotation inventory and structural constraints; models are consulted only at decision nodes, each with the part of the guideline it needs. Each node returns either a *fragment*, a schema-checked annotation from an LLM, or a *decision*, probabilities over candidate annotations from a decision model (TypeSafe, 2026; OpenAI, 2026; Microsoft, 2026). Each step is explicit and logged, allowing a reflector LLM to trace errors to the very steps that produced them and revise the responsible skills or the harness itself. This enables recursive self-improvement (RSI) of the procedure without changing model weights.

To our knowledge, this is the first work to annotate complex schemes step by step along their guidelines, and we show that it pays twice: (1) errors are diagnosable and can be fixed where they arise, and (2) with very little data, RSI makes the procedure much better than the guideline it started from and than a frontier model given the whole guideline.

We evaluate the framework with small LLMs and decision models on Chinese and English UMR parsing (Štěpánek et al., 2026) and PARSEME verbal multiword expressions (Ramisch et al., 2018, 2020). Reflection with a few dozen annotated sen-

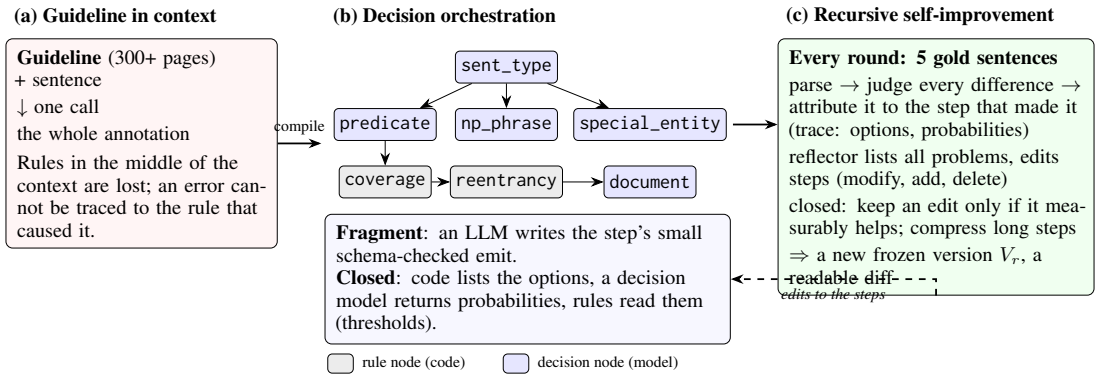


Figure 1: (a) The usual way to give an LLM a codebook: the whole guideline in context. (b) Decision orchestration compiles the guideline into a procedure of small steps, rule nodes and decision nodes, dispatched by a registry; a decision node is either a fragment decision by an LLM or closed questions answered by a decision model. (c) With a handful of gold sentences per round, a reflector edits the steps that made the errors.

tences improves most procedures across the two tasks and languages.

Our contributions are threefold:

- **A traceable framework for annotation and moderation.** Decision orchestration combines code-enforced constraints with local model judgments. Explicit decision traces support moderation through targeted revisions to skills and the harness, enabling recursive self-improvement without changing model weights.
- **Improvement with minimal supervision.** Reflection using only a few dozen sentences outperforms simply-prompted larger model baselines, indicating the framework's great ability in effectively learn from scarce examples.
- **An analysis of how procedures improve and fail.** We trace errors to candidate generation, model judgments and structural rules, and examine the revisions made during reflection. The analysis reveals both concrete opportunities for improvement and limits that local model decisions alone cannot overcome.

These results suggest that LLMs can learn an annotation scheme from its instructions and a few dozen gold sentences, keep improving as new gold arrives, and that the models doing the annotating need not be frontier models: even simple decision models can serve.

2 Related Work

LLMs as annotators LLMs can replace or assist crowd workers on many labeling tasks (Wang et al., 2021; Gilardi et al., 2023; Törnberg, 2025; He et al., 2024; Ziems et al., 2024; Tan et al., 2024), but their quality varies by task and must be validated against human labels (Pangakis et al., 2023). Zero-shot LLM labels often miss a benchmark's specific conventions. Prior work narrows this gap by adding guidelines to prompts and refining them through moderation (Kim et al., 2026), learning guidelines from errors (Pang et al., 2023), or training models to follow guidelines written as code (Sainz et al., 2024; Wang et al., 2023). Yet it remains unclear how a model can apply a long guideline reliably as all of these keep the guideline as one block of context for one model call.

For complex ontologies such as UMR, whose guidelines run to hundreds of pages (Van Gysel et al., 2021; Bonn et al., 2025), that block exceeds what a model reliably uses: performance falls for information in the middle of long contexts (Liu et al., 2024) and drops roughly exponentially with the number of instructions to be followed at once (Harada et al., 2025). Expert-level structured annotation by prompting remains weak (Ettinger et al., 2023; Sun et al., 2024), and supervised parsers need large corpora (Chun and Xue, 2024; Markle et al., 2026). Together, these limitations leave a gap: how to apply long annotation guidelines reliably without a large labeled corpus.

Recursive self-improvement and rule-based reflection Early NLP systems improved annotation by learning rules from gold labels (Brill, 1992, 1995; Ramshaw and Marcus, 1995), but relied on predefined rule templates. LLMs have renewed this idea with more flexible revisions of prompts (Pryzant et al., 2023; Khattab et al., 2024), rule sets (Zhang et al., 2026b; Zhao et al., 2024), or harness codes(Zelikman et al., 2024; Hu et al., 2025; Zhang et al., 2026a). However, their use in expert structured annotation remains underexplored, espe-

cially when annotations must follow long, complex guidelines. The limits of self-correction without external feedback (Huang et al., 2024) motivate a structured harness that constrains revisions and checks whether they improve annotation accuracy.

3 Methodology: Decision Orchestration

Workflow and components A registry runs rule nodes and decision nodes to produce an annotation and an execution trace. On gold data, a judge scores individual annotation units, and a reflector uses the scores and trace to revise the procedure. We represent this system as

$$\begin{aligned} \Pi &= (\mathcal{S}, R, C, M, W, \Theta, J, \mathcal{F}), \\ s_i &= (I_i, O_i, d_i, T_i) \in \mathcal{S}, \\ \mathcal{F} &= (F_{\text{diag}}, F_{\text{valid}}, F_{\text{comp}}). \end{aligned} \tag{1}$$

Here, $\mathcal{S}$ contains decision nodes s_i, each implemented as a skill with input and output schemas I_i, O_i, instructions d_i, and available tools T_i. R is the registry, C the deterministic code, W the worktree, M the annotation and reflection models, and Θ numerical parameters such as thresholds. J is the set of fine-grained judges; $\mathcal{F}$ comprises diagnosis, edit validation and compression.

R executes nodes in ordered groups (*waves*), dispatching a rule node in C or a decision node in $\mathcal{S}$(*emits*), categorizes outputs to worktree W or to next nodes, and assembles the final outputs from the worktree W. At s_i, M evaluates the input under d_i, with supporting information from T_i; C validates or converts its answer to an emit conforming to O_i, using Θ for closed decisions, and assembles the annotation. J scores it against gold in fine-grained aspects, along with a *comprehensive score* defined by the annotation benchmark; $\mathcal{F}$ invokes a reflector in M to locate errors, propose edits to the responsible instructions or code, then validates and compresses the revised procedure to avoid skill instructions from becoming too verbose.

Modes of Decision nodes Decision nodes have two implementations with the same emit. In **fragment mode**, an LLM reads the skill's instructions and input, uses its available tools, and returns a graph fragment in structured outputs. In **closed mode**, the system uses codes to enumerate options and asks choice, yes/no or graded questions. A decision model returns probabilities over these options, which rules convert into the emit using thresholds in Θ. Both modes log inputs and outputs; closed mode also records options and probabilities.

Cascading For closed decisions with confidence lower than a *cascading threshold*, a **cascading** mechanism routes the input to a stronger model that receives the same inputs and replaces the original answer.

Calibration For the *cascading threshold* above, and to account for different decision models' output probabilities for*decision thresholds*, we fit both in 0.1 increments on a small batch of training annoations for a best comprehensive score.

Fine-grained evaluation The judges in J score individual annotation units against gold; execution traces link predictions to the responsible steps, providing targeted feedback for reflection. Task-specific checks and scoring details are given in the Metrics paragraph of Experiments (§4).

Reflection Reflection revises the procedure in three steps: **Diagnose (F_{diag}):** use judge feedback and execution traces to locate errors and propose targeted edits. **Validate (F_{valid}):** check candidate revisions and retain those that satisfy the acceptance criteria. **Compress (F_{comp}):** merge redundant instructions and rules while preserving required options and conventions. The experimental protocol is detailed in §4 (Reflection protocol).

4 Experiments

Tasks We evaluate two structured annotation tasks (Figure 2). **UMR** maps sentences to token-aligned graphs of concepts, roles and attributes, and adds document-level temporal, modal and coreference relations (Van Gysel et al., 2021; Bonn et al., 2025). **PARSEME** identifies verbal multiword expressions (VMWEs), including discontinuous expressions, and assigns each a category (Ramisch et al., 2018, 2020).

Data For UMR, we use Chinese and English data from the DMR 2026 shared task, largely based on UMR 2.1 (Štěpánek et al., 2026). We select 50 training sentences per language in ten batches of five; the test sets contain 100 Chinese and 95 English sentences. We use shared-task tokenization. For PARSEME, we use Chinese edition 1.2 (Ramisch et al., 2020) and English edition 1.1 (Ramisch et al., 2018). Each language has 50 training sentences in ten batches of five (three with a

VMWE and two without per batch) and 100 sampled official test sentences (50 with a VMWE and 50 without). Table 1 lists the decision skills and rule nodes, Figure 3 shows a UMR trace, and Appendix A gives the seed instructions.

Baseline We use GPT-6.1-sol to generate the annotation in a single call per test chunk, given the full guideline, the input sentences (including lemma, part-of-speech and dependency columns for PARSEME) and the required output format. The guideline itself contains examples; we provide no separate gold examples.

Seeding We write the seed procedures solely from the published guidelines before reading any training gold. Neither mode includes worked examples (Appendix A). In fragment mode, decision skills restate the relevant guideline in natural language and return schema-constrained graph fragments or expressions. In closed mode, we encode possible outputs as fixed-choice questions, with guideline tests, thresholds and deterministic branches; the model supplies choices and code assembles the annotation.

Models and batches Each training batch contains five sentences. Fragment-mode parsing uses GPT-6-luna and Claude Haiku 5.5; Claude Sonnet 5.5 is also tried as a fragment decider, for two rounds on UMR and three on PARSEME. Closed-mode parsing uses Jev 1.13 (TypeSafe, 2026), OpenAI Decisions with `gpt-6-luna-decisions` (OpenAI, 2026), and Microsoft-Decision-1 (Microsoft, 2026). Claude Sonnet 5.5 performs reflection. OpenAI Decisions is called through OpenAI's API, Sonnet's UMR decisions through the Claude subscription, and all other models through OpenRouter.

Reflection protocol The reflector reads gold and predicted annotations, and unit-level scores, attributed errors, decision traces and editable files, then proposes additions, replacements or deletions to instructions (Algorithm 1). In closed mode, each edit is applied to a copy and evaluated on the current and all earlier training batches. It is kept if current-batch comprehensive score improves and earlier-batch score falls by at most 0.5, or earlier-batch score improves without a current-batch loss. Rejected edits are returned once with their measured effects. In fragment mode, edits are accepted if they apply and leave the procedure loadable.

Files exceeding $1.5\times$ their seed length are compressed, merging redundant rules while preserving output format parts like options, questions and conventions.

Metrics UMR is scored with the shared task's official juːmætʃ F1 (Štěpánek et al., 2026), where predicted nodes are mapped to gold nodes through their token alignment, and F1 is computed over the triples (concepts, relations, attributes) of mapped nodes, document-level relations included. For diagnosis we use a fine-grained judge (Table 2): it maps nodes with AnCast (Sun and Xue, 2024), breaks the comparison into 25 checks in 11 modules, gives every unit a verdict and every check its own F1, and attributes each predicted unit to the skill call that produced it. PARSEME is scored with the official evaluation suite *MWE-based F1* (exact token set and category), token-based F1, and F1 on expressions unseen in the training data. A PARSEME judge sorts every gold and predicted expression into six kinds: correct, wrong category (same tokens), partial (overlapping tokens), missed without a candidate (no rule proposed its tokens), missed after rejection (a candidate had them), and spurious (no overlap with any gold expression).

SOTA For UMR, we report the best shared-task submission (Štěpánek et al., 2026), Orange/08 (Heinecke and Asadullah, 2026) (36.5 juːmætʃ in Chinese, 22.2 in English), on the full blind test set. We have the test gold but not its predictions, so we cannot re-score it on our chunks. Chun and Xue (2024) and SETUP (Markle et al., 2026) evaluate on UMR 1.0 and 2.0, respectively, and use externally trained components or additional training data; we therefore exclude their scores from this comparison. For PARSEME, we re-score the shared-task systems on our sentences (Chinese 1.2: TRAVIS-mono/-multi; English 1.1: Milos and TRAPACC). On the full test files our scoring reproduces their published F (71.9 for TRAVIS-mono, 33.3 for Milos); on our sample, which has as many sentences with a VMWE as without, the same systems score higher (TRAVIS-multi 81.1, TRAVIS-mono 80.7, Milos 40.5). The best published English 1.1 result (Rohanian et al., 2019) is quoted on the full test set.

(a) UMR: a sentence graph, its alignment, and document-level relations

And$_1$ *you*$_2$ *watch*$_3$ *him*$_4$ *pluck*$_5$ *a*$_6$ *few*$_7$ *pears*$_{8,9}$ (sentence 10 of a narrative)

```
(s10w / watch-01
 :ARG0 (s10p / person
    :refer-person 2nd :refer-number singular)
 :ARG1 (s10p2 / pluck-01
    :ARG0 (s10p3 / person
       :refer-person 3rd :refer-number singular)
    :ARG1 (s10p4 / pear :quant (s10a / a-few))
    :aspect activity)
 :aspect state)
```

```
alignment: s10w 3-3, s10p 2-2, s10p2 5-5,
  s10p3 4-4, s10p4 8-8, s10a 6-7
:temporal ((s8e :after s10w)
       (s10w :overlap s10p2))
:modal ((root :modal author)
     (author :full-affirmative s10w) ...)
:coref ((s8p :same-entity s10p)
     (s9p2 :same-entity s10p4))
```

(b) PARSEME: verbal multiword expressions

After that, he $\underline{made}_6$ *a great* $\underline{effort}_9$ *to be calm.*
$\{6,9\}$: `LVC.full` (light verb + predicative noun, discontinuous)

So Harry had been $\underline{brought}_5$ $\underline{up}_6$ *by his dead mother's sister ...*
$\{5,6\}$: `VPC.full` (the particle changes the verb's meaning)

我 已經 作出$_3$ 決定$_4$ ， 讓 他 發言 。
$\{3,4\}$: `LVC.full`; $\{3\}$: `VPC.semi` (one token: 作 + complement 出)

Categories (EN 1.1): VID, LVC.full, LVC.cause, VPC.full, VPC.semi, IAV;
(ZH 1.2): VID, LVC.full, LVC.cause, VPC.semi, MVC.

Figure 2: Training examples of (a) UMR sentence graphs, token alignments and document-level relations, and (b) PARSEME verbal multiword expressions, which may be discontinuous or overlap.

Scheme	Node	Kind	Responsibility
UMR	`sent_type`	Decision	Classify a span and route it; emit a graph for a leaf.
	`discourse`	Decision	Split a span into clauses and assign their discourse relation.
	`predicate`	Decision	Choose the head, sense, aspect, modal strength, polarity and argument roles.
	`np_phrase`	Decision	Annotate noun-phrase heads, modifiers, roles, quantity and reference attributes.
	`special_entity`	Decision	Annotate names, dates, quantities and ordinals.
	`document`	Decision	Assign temporal, modal and coreference relations.
	`coverage`	Rule	Return uncovered content tokens to sentence routing.
	`antecedent`	Rule	Record participants outside the current span for later merging.
	`reentrancy`	Rule	Merge nodes with matching types and spans and resolve outside participants.
PARSEME	`candidates`	Rule	Propose token groups from forms, lemmas, parts of speech and dependencies.
	`decide`	Decision	Identify and categorize expressions: return token groups in fragment mode, or answer linguistic tests in closed mode.

Table 1: Implementation details of decision and rule nodes.

5 Results

5.1 Decision Orchestration vs. Baseline vs. SOTA

Table 3 compares each procedure with one-shot generation baseline from GPT-6.1-sol given the full guideline. Before seeing gold data, all three closed seeds exceed that baseline by 9–11 F1 points on Chinese UMR (33.2–35.0 vs. 23.9), and are within three points on English UMR. The Chinese PARSEME fragment seeds also start higher (59.5–72.1 vs. 50.0). English PARSEME is the exception: even after reflection, no procedure reaches the baseline's 75.6.

Against shared-task systems, the best reflected procedure on Chinese PARSEME scores 75.9, below the re-scored 1.2 SOTA's 81.1; on English PARSEME all reflected procedures exceed the re-scored 1.1 SOTA's 40.5. Our best UMR scores, 37.6 Chinese and 32.6 English, exceed the published shared-task scores of 36.5 and 22.2.

5.2 Improvement from Reflection

Figure 4 shows every iteration's performance on test data; Appendix Table 7 gives the scores. Reflection improves 16 of the 18 ten-round chains. The largest gains are in closed PARSEME: Jev rises from 37.8 to 63.4 in Chinese and 22.5 to 53.7 in English; OpenAI Decisions rises from 35.1 to 54.5 and 42.0 to 56.6. On UMR, Jev reaches the best Chinese score (37.6), while Haiku reaches the best English score on the full test sample (32.6). Sonnet 5.5 reaches 75.9 on Chinese PARSEME after three rounds and the highest English UMR score, indicating a stronger LLM indeed performs better.

How Improvement Happens In the first Chinese UMR batch, Haiku's fragment procedure produced English concept names for Chinese words. Reflection added the convention that Chinese event and entity concepts retain the source word to the concept decisions; across that round, test juːmætʃ F1 rose from 20.9 to 27.8. By V_{10}, its event-concept check rises from 30.0 to 77.3 and its entity-concept check from 36.3 to 73.8 (Figure 5). The PARSEME reflector likewise sharpened the closed light-verb test, including the rule that 有 is not a light verb. Across Jev's ten Chinese rounds, spurious predictions fall from 80 to 9, correctly categorized expressions rise from 7 to 22, and test F1 rises from 37.8 to 63.4 (Figure 6). These are examples of revised decision criteria; the gains across rounds include other edits.

The reflector lists 9–11 problems per UMR batch and 2–4 per PARSEME batch, and proposes about one edit per problem (Table 4). About half the closed-mode edits survive measurement. Fragment edits are kept when they apply, without a score check: Haiku's Chinese PARSEME peaks at 62.9 in round 3 but ends at 53.2. Retained edits include `exist-91` for X中有Y, the rule that speech content is not discourse, and an inceptive-aspect rule.

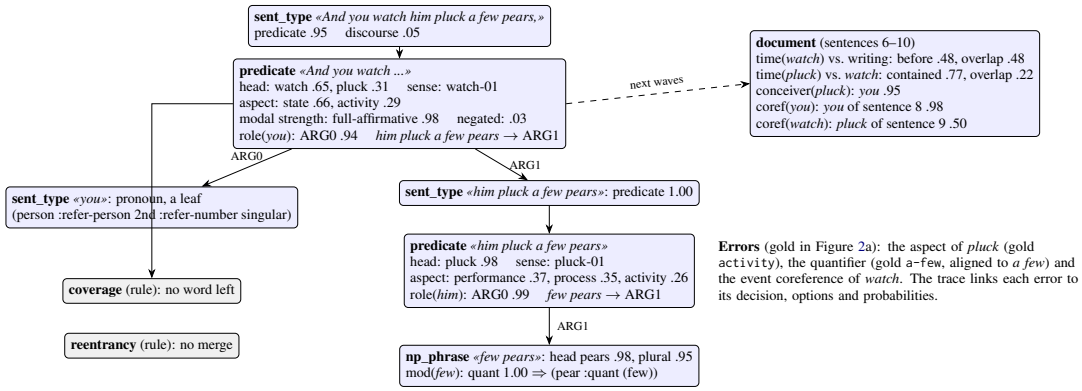


Figure 3: A real trace of the closed-mode UMR seed with Jev. Blue: decision nodes with options and probabilities; grey: rule nodes. The callout identifies decisions that differ from gold (Figure 2a).

Module	Checks	What one unit compares (for matched gold/predicted nodes)	Unit kind
a. nodes	event found, entity found	does a gold event (entity) node have a predicted match; is a predicted one matched	node
	event concept, entity concept	is the concept the same	node pair
	granularity	is the node aligned to the same words (one node for several gold nodes or vice versa)	span
b. sense	event sense	same PropBank roleset number, once the concept is right	node pair
c. attributes	aspect, modal strength, polarity, refer, other	same value of the attribute; an attribute only one side has is missing or extra	attribute
d. arguments	core args of events / of abstract predicates; peripheral args of events / of entities	how far apart the argument sets of a matched head are (credit = share that agrees)	head
e. role labels	core, peripheral	same role label on a matched edge	edge
f. names	name tokens, wiki	same `:op` tokens of a matched name; same wiki title	entity
g. reentrancy	reentrancy	is an edge back to an already declared node also in gold	edge
h. temporal	temporal dependency	a child node's temporal parent(s) and relation(s)	child
i. modal	modal dependency	an event's conceiver (parent) and modal strength (label)	child
j. coreference	chains, subsets	is a mention linked to the same other mentions as in gold; subset relations	mention
k. format	document format	are the predicted document-level relations well formed (no gold needed)	relation

Table 2: The fine-grained UMR judge. Every unit gets a verdict: correct (credit 1), partial (0.5, or the check's own credit), wrong, missing or extra (0). Recall counts the gold side (correct, partial, wrong, missing), precision the predicted side (correct, partial, wrong, extra), and each check reports its F1. Modules a–f never see reentrant edges, so module g's errors do not move them. Each predicted unit is attributed to the skill call that wrote it.

	UMR (ju:mætʃ F1)		PARSEME (MWE F1)	
System	Chinese	English	Chinese	English
Best shared-task system	36.5	22.2	**81.1**	40.5
GPT-6.1-sol + full guideline	23.9	30.2	50.0	**75.6**
fragment: GPT-6-luna, seed V_0	18.9	24.4	62.5	62.3
V_{10}	28.4	34.3	72.5	58.4
fragment: Haiku 5.5, seed V_0	20.9	28.8	59.5	57.6
V_{10}	33.2	32.6	53.2	58.2
fragment: Sonnet 5.5, seed V_0	29.7	39.7	72.1	58.5
V_{10}	35.0	**42.5**	**75.9**	56.5
closed: Jev 1.13, seed V_0	33.9	29.8	37.8	22.5
V_{10}	**37.6**	31.8	63.4	53.7
closed: OpenAI Decisions, seed V_0	33.2	27.6	35.1	42.0
V_{10}	35.6	28.4	54.5	56.6
closed: Microsoft-Decision-1, seed V_0	35.0	29.0	36.7	25.9
V_{10}	35.8	31.0	46.0	43.5

Table 3: Test F1 on UMR (official ju:mætʃ) and PARSEME (MWE F1). V_0 is the guideline-derived seed; V_{10} follows ten reflection rounds on 50 gold sentences. †Published UMR shared-task scores use whole documents. PARSEME shared-task systems are re-scored on our test sentences.

Task	Decisions	problems	edits	kept	compr.	size V_0	size V_{10}
UMR zh	Haiku 5.5	108	57	55	28	36.8k	50.8k
	Jev 1.13	108	109	54	0	70.5k	73.7k
	OpenAI Decisions	110	102	31	0	70.5k	73.3k
	Microsoft-Decision-1	114	106	54	0	70.5k	74.8k
UMR en	Haiku 5.5	100	59	58	17	36.8k	51.5k
	Jev 1.13	87	89	35	0	70.5k	72.6k
	OpenAI Decisions	78	81	27	0	70.5k	71.5k
	Microsoft-Decision-1	85	71	35	0	70.5k	72.7k
PARSEME zh	GPT-6-luna	26	20	18	12	4.5k	8.0k
	Haiku 5.5	29	18	18	9	4.5k	7.5k
	Jev 1.13	37	38	16	2	8.0k	9.6k
	OpenAI Decisions	37	59	13	0	8.0k	9.3k
	Microsoft-Decision-1	38	46	13	0	8.0k	9.2k
PARSEME en	GPT-6-luna	24	18	18	8	4.7k	8.1k
	Haiku 5.5	26	21	21	9	4.7k	8.1k
	Jev 1.13	33	36	13	0	8.1k	9.7k
	OpenAI Decisions	40	50	14	2	8.1k	10.0k
	Microsoft-Decision-1	38	31	8	0	8.1k	9.2k

Table 4: Ten rounds of reflection: problems the reflector listed, edits tried (with closed-mode retries), edits kept, compressions, and the size of the editable procedure (characters; closed mode includes code).

5.3 Fragment vs. Closed Decisions

Figure 5 shows that at V_{10}, closed decisions do better where code constrains the structure: Chinese node granularity is 92.9 vs. 85.8 for fragments, and person and number is 49.0 vs. 23.2. Fragments do better on open judgments: event arguments are 45.6 vs. 35.7 and coreference 16.4 vs. 12.7. The gains are uneven: Haiku's Chinese core arguments rise by 23.4 points while Jev's fall by 2.1; Jev's English event concepts lag Haiku's (55.6 vs. 72.5). The trace attributes 51 of Jev's 212 wrong English concepts to inflected forms such as *stolen* and *thought* that its rule-based lemmatizer fails to reduce.

On PARSEME (Figure 6), some closed seeds over-accept candidates: Jev and Microsoft make 78–123 spurious predictions against 57–59 gold ex-

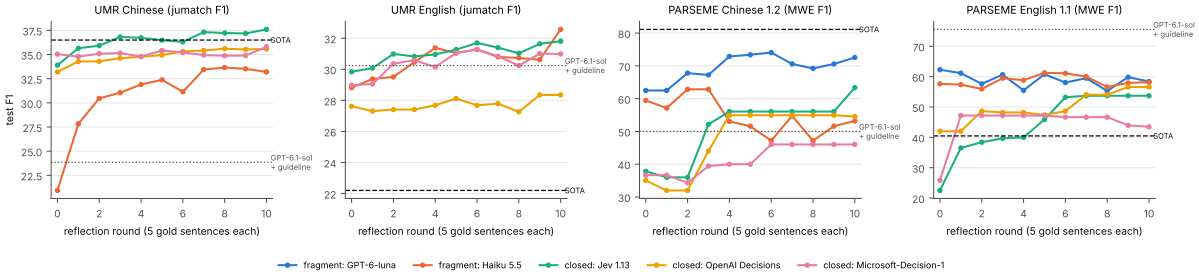


Figure 4: Test F1 of every iteration $V_0, \ldots, V_{10}$, and round r means $5r$ gold sentences have been reflected upon. Dotted is baseline: GPT-6.1-sol with the whole guideline. Dashed is SOTA: best shared-task system.

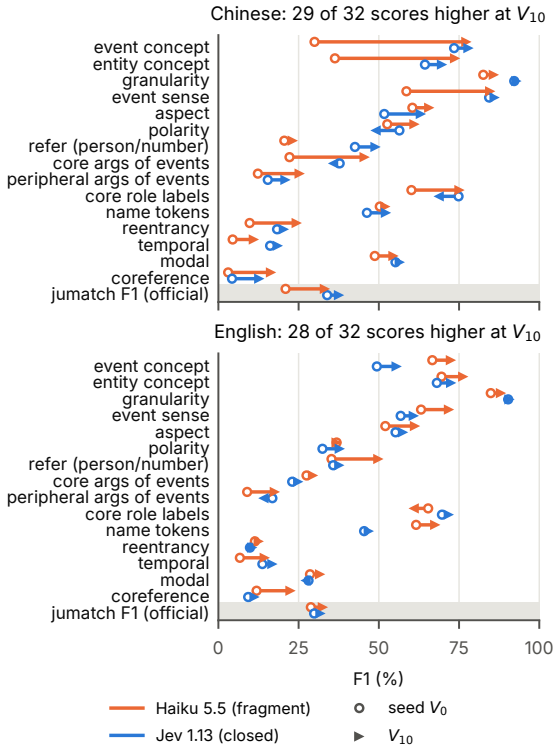


Figure 5: Reflection improvement of the fine-grained UMR judge on the test set: F1 (%) of each check, before and after reflection. Each row is one check, with one line per decider: orange, Haiku 5.5 making fragment decisions; blue, Jev 1.13 making closed decisions. The open circle is the seed V_0, the arrowhead V_{10}; an arrow pointing right means reflection raised the score, pointing left that it lowered it, and a lone marker that it barely changed. Chinese above, English below; panel titles count the rising arrows. Shaded row: the official ju:mætʃ F1. Selected checks; numbers in Appendix Table 8, all 25 checks in the released results.

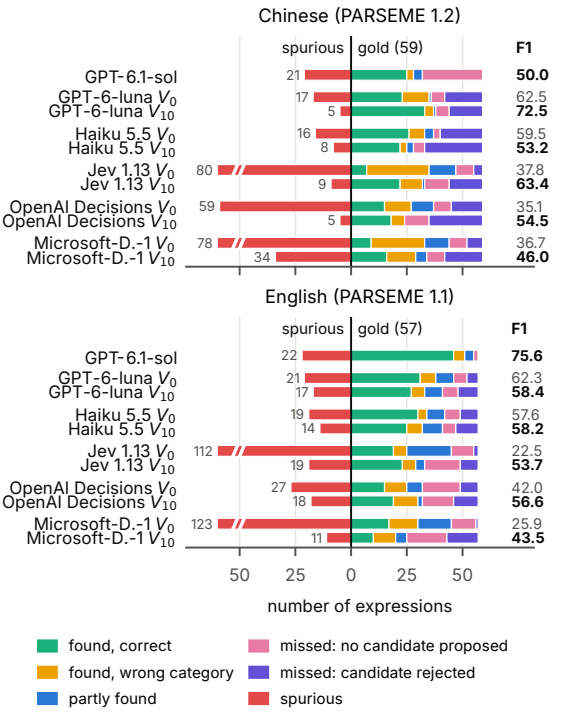


Figure 6: What each PARSEME procedure does with the test set, seed V_0 and V_{10}; Chinese above, English below. Right of 0: the gold expressions. Green (found, correct): predicted with the same tokens and category. Orange (found, wrong category): same tokens, other category. Blue (partly found): the prediction shares some but not all tokens. Pink (missed: no candidate proposed): no rule of the procedure proposed the tokens, so no decision was ever asked. Purple (missed: candidate rejected): a candidate with these tokens was proposed and then rejected by a decision or threshold. Left of 0, red (spurious): predictions that overlap no gold expression, with their count beside the bar; bars longer than 60 are cut (white break mark). A better procedure has more green on the right and a shorter red bar on the left. Right column: MWE-based F1 (category-blind, as in the official script), bold for V_{10} and the baseline (GPT-6.1-sol with the guideline, one bar). Numbers in Appendix Table 9.

pressions. Reflection reduces spurious predictions in all six closed chains (Jev: 80 to 9 Chinese, 112 to 19 English; Microsoft: 123 to 11 English). It can also reject good candidates: Microsoft's English correct predictions fall from 17 to 10 and missed gold expressions rise from 12 to 32, although its F1 improves from 25.9 to 43.5. Closed procedures

	UMR zh		UMR en		PARSEME zh		PARSEME en	
Procedure	raw	cal.	raw	cal.	raw	cal.	raw	cal.
Jev 1.13 V_0	33.9	34.2	29.8	30.2	37.8	39.2	22.5	27.1
Jev 1.13 V_{10}	37.6	37.4	31.8	31.8	63.4	61.4	53.7	49.5
Microsoft-Decision-1 V_0	35.0	35.8	29.0	29.0	36.7	35.0	25.9	29.9
Microsoft-Decision-1 V_{10}	35.8	36.0	31.0	31.1	46.0	46.0	43.5	43.5

Table 5: Closed procedures with their thresholds as written (raw) and calibrated on the 50 training sentences (cal.).

additionally miss 8–18 expressions at V_{10} because no rule proposes their tokens, against 3–7 for fragments, which may return expressions outside the candidate list.

5.4 Calibration

Table 5 compares each closed procedure with its thresholds as written and after calibration on the 50 training sentences. Calibration helps six of eight closed seeds but changes reflected UMR procedures by at most 0.2 points and can hurt PARSEME (Jev Chinese: 63.4 to 61.4). Reflection has already adjusted the decisions, and refitting thresholds on so little data can overfit. The learned procedure also transfers between deciders: Jev's V_{10}, answered by Microsoft-Decision-1, beats Microsoft's own V_{10} in three of four cases (Chinese PARSEME: 57.1 vs. 46.0, or 62.0 after calibration). Cascading uncertain decisions to a small LLM is reported in Appendix C.

6 Discussion

Diagnosability Each prediction records its step, candidate options and decision scores, so an error points to a particular criterion. In Chinese UMR, the concept checks exposed English names for Chinese words, so that the reflector added a source-word convention to the fragment concept instructions, and test F1 rose from 20.9 to 27.8 in the first round (§5.2). In Chinese PARSEME, false light-verb decisions prompted a stricter closed light-verb test, including the rule that 有 is not a light verb; Jev's spurious predictions fell from 80 to 9 over ten rounds. The trace also locates errors reflection has not fixed: 51 of Jev's 212 wrong English event concepts are inflected forms left unchanged by its rule-based lemmatizer (§5.3). This distinguishes a decision rule that needs revision from code that needs new options.

Low-cost decision models For 100 Chinese UMR sentences, parsing costs about \$0.1–0.3 with a decision model, \$1.9 with Haiku, and \$7.1 with GPT-6.1-sol using the full guideline (Appendix Table 10). Cheap closed decisions make it practical to scale. The performance is competitive. Jev gives the best Chinese UMR procedure (37.6 ju:mætʃ F1). The coverage limit is visible in PARSEME: a closed decider cannot recover an expression absent from its candidate list, while fragments can. The transfer result in §5.4 indicates that one revised procedure can serve another decision model, which shows great potential for decision models on this task.

Limitation of reflection All problems coming together during reflection do not guarantee steady improvement. Fragment edits can drift such as Haiku's Chinese PARSEME score peaks at 62.9 at V_3 and ends at 53.2. Even measured closed edits are selected on small batches, and recalibrating their thresholds on all 50 training sentences can lower test F1. Reflection revises rules within the existing decomposition determined by codes, and it cannot add candidates that the closed PARSEME enumerator never proposes without a code change, which may cause problems. It also does not close the gap to the one-shot model on English PARSEME, where the task fits the frontier model's prior knowledge.

7 Conclusion

We compile annotation guidelines into step-by-step procedures and use them to annotate UMR and PARSEME. A fine-grained evaluator traces errors to individual decisions, giving reflection a specific place to revise; this improves 16 of 18 ten-round runs. Without supervised fine-tuning, our reflected procedures surpass the best shared-task system on English PARSEME on the same sentences, and our best UMR scores exceed the published shared-task results on smaller evaluation units. Chinese PARSEME remains below its shared-task best. These results show the potential of building annotators directly from guidelines: decisions and revisions are inspectable, making the procedure easy to debug and improve as more gold annotations arrive. Cheap decision models can carry out the annotation at scale. We see this as a natural first step for future annotation projects, to develop a guideline as an executable procedure and then refine it through fine-grained evaluation as the scheme matures.

Limitations

Due to budget limit, we can only run test sets for 95–100 sentences per task and language and training is 50 sentences. With 57–59 gold expressions per PARSEME test set, one expression is worth about one point of F, so differences of a few points there should not be over-read. We did not repeat chains with other document orders to measure the variance of reflection. Latest PARSEME 2.0 has no Chinese data and only an English trial set, so we use editions 1.2 (Chinese) and 1.1 (English), where the English 1.1 systems predate pretrained LLMs we use. The seeds were written by the authors with an LLM's help from the guidelines, so their quality affects every result and we have not measured the effort they take. Closed and fragment seeds share the decomposition, so the comparison between modes is not a comparison of decompositions. Fragment-mode edits are accepted unmeasured and may drift, and the same reflector (Claude Sonnet 5.5) was used throughout, so different reflectors are not measured. All models are commercial services whose behavior may change. We will release the predictions, the procedures at every version, and the cached answers that their terms permit.

Ethics Statement

We use the UMR shared-task data and PARSEME 1.1 and 1.2 under their licenses (PARSEME 1.2 Chinese: CC BY-NC-SA 4.0) and collect no data from people. The method aims to make the first annotations of a new scheme cheaper and easier to check; it could reduce demand for annotation work, and we see it rather as a way to direct annotators to the decisions that need them. Automatic annotations should be checked before they are used as gold. Generative AI assisted the implementation, the running of the experiments and the editing of this paper; the authors are responsible for all content.

References

Xuefeng Bai, Yulong Chen, and Yue Zhang. 2022. Graph pre-training for AMR parsing and generation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6001–6015, Dublin, Ireland. Association for Computational Linguistics.

Michele Bevilacqua, Rexhina Blloshmi, and Roberto Navigli. 2021. One SPRING to rule them both: Symmetric AMR semantic parsing and generation without a complex pipeline. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(14):12564–12573.

Julia Bonn, Claire Bonial, Matt Buchholz, Hsiao-Jung Cheng, Alvin Chen, Ching-wen Chen, Andrew Cowell, William Croft, Lukas Denk, Ahmed Elsayed, Eva Fučíková, Federica Gamba, Carlos Gomez, Jan Hajič, Eva Hajičová, Jiří Havelka, Loden Havenmeier, Ath Kilgore, Veronika Kolářová, and 40 others. 2025. Uniform meaning representation 2.0.

Eric Brill. 1992. A simple rule-based part of speech tagger. In *Proceedings of the Third Conference on Applied Natural Language Processing*, pages 152–155, Trento, Italy. Association for Computational Linguistics.

Eric Brill. 1995. Transformation-based error-driven learning and natural language processing: A case study in part-of-speech tagging. *Computational Linguistics*, 21(4):543–565.

Jayeol Chun and Nianwen Xue. 2024. Uniform meaning representation parsing as a pipelined approach. In *Proceedings of TextGraphs-17: Graph-based Methods for Natural Language Processing*, pages 40–52, Bangkok, Thailand. Association for Computational Linguistics.

Allyson Ettinger, Jena D. Hwang, Valentina Pyatkin, Chandra Bhagavatula, and Yejin Choi. 2023. "you are an expert linguistic annotator": Limits of LLMs as analyzers of Abstract Meaning Representation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 8250–8263, Singapore. Association for Computational Linguistics.

Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. 2023. ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, 120(30):e2305016120.

Keno Harada, Yudai Yamazaki, Masachika Taniguchi, Edison Marrese-Taylor, Takeshi Kojima, Yusuke Iwasawa, and Yutaka Matsuo. 2025. When instructions multiply: Measuring and estimating LLM capabilities of multiple instructions following. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 16506–16526, Suzhou, China. Association for Computational Linguistics.

Xingwei He, Zhenghao Lin, Yeyun Gong, A-Long Jin, Hang Zhang, Chen Lin, Jian Jiao, Siu Ming Yiu, Nan Duan, and Weizhu Chen. 2024. AnnoLLM: Making large language models to be better crowdsourced annotators. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, pages 165–190, Mexico City, Mexico. Association for Computational Linguistics.

Johannes Heinecke and Munshi Asadullah. 2026. Orange @ UMR parsing shared task. In *Proceedings of the Seventh International Workshop on Designing Meaning Representations (DMR 2026) @ LREC*

2026, pages 148–154, Palma, Mallorca, Spain. ELRA Language Resources Association.

Shengran Hu, Cong Lu, and Jeff Clune. 2025. Automated design of agentic systems. In *The Thirteenth International Conference on Learning Representations*.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large language models cannot self-correct reasoning yet. In *The Twelfth International Conference on Learning Representations*.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan A, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In *The Twelfth International Conference on Learning Representations*.

Kon Woo Kim, Jin-Dong Kim, and Akiko Aizawa. 2026. Refining and reusing annotation guidelines for LLM annotation. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 37951–37964, San Diego, California, United States. Association for Computational Linguistics.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

Emma Markle, Javier Gutierrez Bach, and Shira Wein. 2026. SETUP: Sentence-level english-to-uniform meaning representation parser. In *Proceedings of the Fifteenth Language Resources and Evaluation Conference*, pages 9710–9721.

Microsoft. 2026. Deploy and use Microsoft-Decision-1 in Microsoft Foundry. https://learn.microsoft.com/en-us/azure/foundry/foundry-models/how-to/use-foundry-models-microsoft-decision. Accessed October 2026.

OpenAI. 2026. Decisions. OpenAI API documentation, https://developers.openai.com/api/docs/guides/decisions. Accessed October 2026.

Chaoxu Pang, Yixuan Cao, Qiang Ding, and Ping Luo. 2023. Guideline learning for in-context information extraction. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 15372–15389, Singapore. Association for Computational Linguistics.

Nicholas Pangakis, Samuel Wolken, and Neil Fasching. 2023. Automated annotation with generative AI requires validation. *arXiv preprint arXiv:2306.00176*.

Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. 2023. Automatic prompt optimization with "gradient descent" and beam search. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7957–7968, Singapore. Association for Computational Linguistics.

Carlos Ramisch, Silvio Ricardo Cordeiro, Agata Savary, Veronika Vincze, Verginica Barbu Mititelu, Archna Bhatia, Maja Buljan, Marie Candito, Polona Gantar, Voula Giouli, Tunga Güngör, Abdelati Hawwari, Uxoa Iñurrieta, Jolanta Kovalevskaitė, Simon Krek, Timm Lichte, Chaya Liebeskind, Johanna Monti, Carla Parra Escartín, and 6 others. 2018. Edition 1.1 of the PARSEME shared task on automatic identification of verbal multiword expressions. In *Proceedings of the Joint Workshop on Linguistic Annotation, Multiword Expressions and Constructions (LAW-MWE-CxG-2018)*, pages 222–240, Santa Fe, New Mexico, USA. Association for Computational Linguistics.

Carlos Ramisch, Agata Savary, Bruno Guillaume, Jakub Waszczuk, Marie Candito, Ashwini Vaidya, Verginica Barbu Mititelu, Archna Bhatia, Uxoa Iñurrieta, Voula Giouli, Tunga Güngör, Menghan Jiang, Timm Lichte, Chaya Liebeskind, Johanna Monti, Renata Ramisch, Sara Stymne, Abigail Walsh, and Hongzhi Xu. 2020. Edition 1.2 of the PARSEME shared task on semi-supervised identification of verbal multiword expressions. In *Proceedings of the Joint Workshop on Multiword Expressions and Electronic Lexicons*, pages 107–118. Association for Computational Linguistics.

Lance Ramshaw and Mitch Marcus. 1995. Text chunking using transformation-based learning. In *Third Workshop on Very Large Corpora*.

Omid Rohanian, Shiva Taslimipoor, Samaneh Kouchaki, Le An Ha, and Ruslan Mitkov. 2019. Bridging the gap: Attending to discontinuity in identification of multiword expressions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2692–2698, Minneapolis, Minnesota. Association for Computational Linguistics.

Oscar Sainz, Iker García-Ferrero, Rodrigo Agerri, Oier Lopez de Lacalle, German Rigau, and Eneko Agirre. 2024. GoLLIE: Annotation guidelines improve zero-shot information-extraction. In *The Twelfth International Conference on Learning Representations*.

Jan Štěpánek, Daniel Zeman, Marketa Lopatkova, Federica Gamba, Hana Hledíková, and Nianwen Xue. 2026. First shared task on UMR parsing. In *Proceedings of the Seventh International Workshop on Designing Meaning Representations (DMR 2026) @ LREC 2026*, pages 136–147, Palma, Mallorca, Spain. ELRA Language Resources Association.

Haibo Sun and Nianwen Xue. 2024. Anchor and broadcast: An efficient concept alignment approach for

evaluation of semantic graphs. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 1052–1062, Torino, Italia. ELRA and ICCL.

Haibo Sun, Nianwen Xue, Jin Zhao, Liulu Yue, Yao Sun, Keer Xu, and Jiawei Wu. 2024. Chinese UMR annotation: Can LLMs help? In *Proceedings of the Fifth International Workshop on Designing Meaning Representations @ LREC-COLING 2024*, pages 131–139.

Zhen Tan, Dawei Li, Song Wang, Alimohammad Beigi, Bohan Jiang, Amrita Bhattacharjee, Mansooreh Karami, Jundong Li, Lu Cheng, and Huan Liu. 2024. Large language models for data annotation and synthesis: A survey. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 930–957, Miami, Florida, USA. Association for Computational Linguistics.

Petter Törnberg. 2025. Large language models outperform expert coders and supervised classifiers at annotating political social media messages. *Social Science Computer Review*, 43(6):1181–1195.

TypeSafe. 2026. Jev API reference. `https://docs.typesafe.ai/api`. Accessed October 2026.

Jens E. L. Van Gysel, Meagan Vigus, Jayeol Chun, Kenneth Lai, Sarah Moeller, Jiarui Yao, Tim O'Gorman, Andrew Cowell, William Croft, Chu-Ren Huang, Jan Hajič, James H. Martin, Stephan Oepen, Martha Palmer, James Pustejovsky, Rosa Vallejos, and Nianwen Xue. 2021. Designing a uniform meaning representation for natural language processing. *KI - Künstliche Intelligenz*, 35(3-4):343–360.

Shuohang Wang, Yang Liu, Yichong Xu, Chenguang Zhu, and Michael Zeng. 2021. Want to reduce labeling cost? GPT-3 can help. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 4195–4205, Punta Cana, Dominican Republic. Association for Computational Linguistics.

Xingyao Wang, Sha Li, and Heng Ji. 2023. Code4Struct: Code generation for few-shot event structure prediction. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3640–3663, Toronto, Canada. Association for Computational Linguistics.

Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. 2024. Self-taught optimizer (STOP): Recursively self-improving code generation. In *First Conference on Language Modeling*.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. 2026a. Darwin Gödel machine: Open-ended evolution of self-improving agents. In *The Fourteenth International Conference on Learning Representations*.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2026b. Agentic context engineering: Evolving contexts for self-improving language models. In *The Fourteenth International Conference on Learning Representations*.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(17):19632–19642.

Caleb Ziems, William Held, Omar Shaikh, Jiaao Chen, Zhehao Zhang, and Diyi Yang. 2024. Can large language models transform computational social science? *Computational Linguistics*, 50(1):237–291.

A The seed procedures

All seeds were written from the guidelines before any training sentence was read, and contain no example: no annotated sentence, no example word in an option description, and every fragment skill's `reference/example.md` is empty. They are in `experiment/orchestration-v2/seeds/`.

UMR. Both modes share the registry: the sentence wave (`sent_type`, `discourse`, `predicate`, `np_phrase`, `special_entity`, and the rule nodes `coverage` and `antecedent`), the rule wave `reentrancy`, and the `document` wave. Each decision skill restates one part of the guidelines: `sent_type` the criteria for events (Part 3-1-1: processes in any packaging, states and entities only when predicated) and the kinds of pieces; `discourse` the lattice of discourse relations (3-1-6); `predicate` light verbs (3-1-3-5), the abstract predicates of non-verbal clauses and their roles (Tables 4 and 8), word senses (3-1-4), participant and non-participant roles (3-2), the aspect decision procedure (3-3-1: event nominals, then habitual, state, activity, endeavor, performance), polarity (3-3-3) and modal strength (4-3-1); `np_phrase` inverse roles for relative clauses, participant nominalizations and relational nouns (3-2-1-3), modifiers (3-2-2-2), quantity and unit (2-2-5) and person and number (3-3-5); `special_entity` the named-entity types of Table 5, dates and quantities; `document` the temporal superstructure and relations (4-2), the modal dependency (4-3) and coreference (4-1). In fragment mode each is a `skill.md` of 2.5–6.5k characters (35k in all) with the output schema; in closed mode each is a Python file that builds the questions and rules, with the model-facing text in a `prompts.yaml`. Function-word, pronoun and nu-

meral tables are closed-class grammar, not annotation decisions.

PARSEME. A rule node proposes candidates from the input's dependency tree: in English a verb with its particle (`compound:prt`), with the head noun of its object or oblique, or with the adposition of its oblique; in Chinese a multi-character verb token, a verb with the verb or adjective right after it that depends on it, a verb with its object, and four-character tokens. In closed mode, each candidate is asked the yes/no tests of its kind from the guidelines' decision trees (LVC.0–LVC.5, VID.1–VID.5, VPC.1–VPC.3, IAV, and the Chinese MVC tree), and rules walk the tree on the answers with one threshold. In fragment mode the LLM reads a `decide.md` that restates the categories and tests, sees the candidates, and returns the expressions of the sentence, which may include expressions the rules did not propose.

B Reflection

Algorithm 1 Reflection on five gold sentences per round.

Require: seed V_0, training batches $B_1, \ldots, B_{10}$, mode $\in$ {closed, fragment}
for $r = 1, \ldots, 10$ **do**
  $V \leftarrow V_{r-1}$; parse B_r with V; judge; attribute each difference to a step
  $P, E \leftarrow$ REFLECTOR(gold, prediction, differences, decisions, files of V, history)
  for each edit $e \in E$ **do**
    if e applies, the procedure loads, and e copies no test text **then**
      if mode = closed **then**
        $\Delta_b, \Delta_g \leftarrow$ change of F on B_r and on $B_1..B_{r-1}$ with e
        keep e iff $(\Delta_b > 0 \wedge \Delta_g \geq -0.5) \vee (\Delta_g > 0 \wedge \Delta_b \geq 0)$
      else keep e
      end if
    end if
  end for
  closed: send rejected edits back once with their measured effect; try the rewrites
  for each file f longer than $1.5\times$ its seed **do**
    rewrite f shorter (reflector); closed: keep it iff no set falls by more than 0.5
  end for
  $V_r \leftarrow V$ (frozen; tested on the test set)
end for

The reflector's instructions (abridged): the procedure follows a written guideline, code cuts, enumerates and assembles, decision nodes judge; list every problem you can find, however small, with its cause in the procedure; propose edits (`replace`, `insert_after`, `delete`, each an exact anchor in one file) that fix them on these sentences and on unseen text; prefer general rules and better option descriptions to special cases; keep every file valid; when you add a rule, delete or merge what it makes redundant. The closed-mode variant lists the useful kinds of edits (the right answer among the options with a rule branch; sharper option descriptions; questions added, split or removed; thresholds; conventions written into rules; code that cuts or assembles wrongly), the fragment-mode variant its own (conventions, misleading instructions removed, explicit decision procedures, output format). The full prompts are in `code/reflect.py`.

C Cascading uncertain decisions to a small LLM

Method. Every closed answer carries a confidence $c = \max_o p(o)$. In the cascade, the questions a decision model answers with $c < \tau$ are asked again, with the same state and options, of a small LLM (Claude Haiku 5.5) that must pick one of the options; its pick replaces the decision model's answer, and nothing else changes. τ is chosen on the 50 training sentences among 0.6, 0.7 and 0.8, and the cascade is used only if some τ improves the training score. We ran it for Jev only, on its seed V_0 and its final V_{10}; reflection itself ran without the cascade.

	UMR (ju:mætʃ F1)		PARSEME (MWE F1)	
	Chinese	English	Chinese	English
Training sentences, Jev V_{10}				
no cascade	37.5	38.5	88.2	76.7
$\tau = 0.6$	37.0	38.0	82.2	64.6
$\tau = 0.7$	37.2	38.2	77.3	73.5
$\tau = 0.8$	37.8	38.8	78.5	69.6
selected τ	0.8	0.8	none	none
Test sentences				
Jev V_0	33.9	29.8	37.8	22.5
Jev V_0 + cascade	35.5	31.4	39.8	25.8
Jev V_{10}	37.6	31.8	63.4	53.7
Jev V_{10} + cascade	38.1	32.6	53.4	44.1

Table 6: The cascade from Jev to Haiku 5.5. Training: Jev's V_{10} on the 50 training sentences without the cascade and at three values of τ. Test: Jev's V_0 and V_{10} without and with the cascade, at the selected τ (UMR: 0.8) or, where no τ helped on the training sentences (PARSEME), at $\tau = 0.7$.

Results. Table 6 gives the results. On PARSEME the cascade hurts. On the training sentences no τ improved Jev's V_{10} (English 76.7 without the cascade, 64.6–73.5 with it; Chinese 88.2 against 77.3–82.2), and at $\tau = 0.7$ the test score falls too (English 53.7 $\rightarrow$ 44.1, Chinese 63.4 $\rightarrow$ 53.4), while

on the seeds it helps a little (37.8 → 39.8, 22.5 → 25.8). Once reflection has tuned PARSEME's rules to Jev's answers, replacing the hard ones with another model's breaks that fit. On UMR, whose closed questions are more open (which concept, which role), the training sentences select $\tau = 0.8$ in both languages (Chinese 37.5 → 37.8, English 38.5 → 38.8), and the cascade adds a little on the test set too: Chinese 37.6 → 38.1 and English 31.8 → 32.6, for $0.18–0.67 per hundred sentences, still a fraction of Haiku's own cost as a fragment decider. On the seeds it adds 1.6 points in both languages (33.9 → 35.5, 29.8 → 31.4). The cascade was not tried with the other decision models, with other escalation targets, or during reflection, so these results describe one configuration only.

D Data details

UMR. Splits are frozen by `code/splits.py` (seed 20261012) in `experiment/orchestration-v2/data/`. Chinese training: one batch of five consecutive sentences from each of ten training documents among `chinese_umr-0001–0018`; Chinese test: 20 chunks of five consecutive sentences from the 13 blind-test documents with gold (`0021–0024`, `0027–0035`), at least one chunk per document. English training: five, three and two batches from `english_umr-0004`, `0578` and `0579`; English test: all 95 sentences of `english_umr-0005–0008` in 20 chunks. Input tokens are the gold `Words` lines, as in the shared task. Gold is rewritten in the strict format the official scorer reads (one graph per sentence, alignments restricted to graph variables, document-level relations restricted to the chunk). Scoring: `compare_umr.pl -tsv`, with and without `-no-document-level`.

PARSEME. Chinese: PARSEME 1.2 `ZH`; English: PARSEME 1.1 `EN`. Training batches: three sentences with at least one VMWE and two without, ten batches. Test: 50 sentences with and 50 without a VMWE, drawn from the official test file. Unseen F uses a training file reduced to the sentences with a VMWE, which gives the same seen/unseen split as the full file and runs in seconds. The shared-task systems' released predictions are re-scored on the same 100 sentences with the official `evaluate.py`.

Guidelines. The UMR guidelines (`umr4nlp/umr-guidelines`, `guidelines.md`, 366k characters) and the PARSEME 1.2 guidelines (335k characters) are given to the baseline in full and were the only source of the seeds.

E All test scores

F Fine-grained judge results

G Costs

H Earlier experiments: fragment decisions by Claude Sonnet 5.5

Before the experiments of the main text, we ran the fragment-mode UMR procedure with a stronger parser, Claude Sonnet 5.5, on Chinese UMR, with an earlier hand-written seed that contained worked examples, an earlier test protocol (whole documents; AnCast++ instead of the shared task's metric) and unmeasured reflection by Claude Opus 5.5 on one whole training document per round. These numbers are not comparable with the main text (different seed, test documents, metric and gold exposure) and are reported for completeness.

Table 11 gives the test scores and Table 12 the training documents. One training document raised the first test set from 45.3 to 50.8 F and a second to 52.6, with temporal relations improving most; third and fourth documents left the test scores flat. Unmeasured reflection accepted 440 of 441 edits and the skills grew from 41k to 156k characters in ten rounds, which motivated the length compression of the main text.

Task	Decisions	V_0	V_1	V_2	V_3	V_4	V_5	V_6	V_7	V_8	V_9	V_{10}
UMR zh	Haiku 5.5	20.9	27.8	30.5	31.1	31.9	32.4	31.1	33.5	33.6	33.5	33.2
	Jev 1.13	33.9	35.6	35.9	36.8	36.7	36.5	36.3	37.3	37.2	37.2	37.6
	OpenAI Decisions	33.2	34.3	34.3	34.6	34.8	34.9	35.3	35.4	35.6	35.5	35.6
	Microsoft-Decision-1	35.0	34.8	35.1	35.1	34.8	35.4	35.2	35.0	34.9	34.9	35.8
UMR en	Haiku 5.5	28.8	29.4	29.5	30.5	31.4	31.1	31.3	30.8	30.7	30.6	32.6
	Jev 1.13	29.8	30.1	31.0	30.8	31.0	31.3	31.7	31.4	31.0	31.6	31.8
	OpenAI Decisions	27.6	27.3	27.4	27.4	27.7	28.1	27.7	27.8	27.3	28.4	28.4
	Microsoft-Decision-1	29.0	29.1	30.4	30.6	30.1	31.1	31.3	30.8	30.2	31.0	31.0
PARSEME zh	GPT-6-luna	62.5	62.5	67.8	67.2	72.9	73.4	74.1	70.6	69.2	70.6	72.5
	Haiku 5.5	59.5	57.1	62.9	62.9	53.1	51.6	47.2	54.7	47.2	51.6	53.2
	Jev 1.13	37.8	36.0	36.0	52.2	56.1	56.1	56.1	56.1	56.1	56.1	63.4
	OpenAI Decisions	35.1	32.0	32.0	44.0	54.9	54.9	54.9	54.9	54.9	54.9	54.5
	Microsoft-Decision-1	36.7	36.7	34.3	39.4	40.0	40.0	46.0	46.0	46.0	46.0	46.0
PARSEME en	GPT-6-luna	62.3	61.2	57.6	60.7	55.5	60.8	58.1	59.5	55.3	59.8	58.4
	Haiku 5.5	57.6	57.4	55.9	59.5	58.8	61.3	61.1	60.0	56.6	57.9	58.2
	Jev 1.13	22.5	36.5	38.4	39.7	40.0	45.9	53.2	53.7	53.7	53.7	53.7
	OpenAI Decisions	42.0	42.0	48.6	48.2	48.2	47.4	48.6	54.0	54.0	56.6	56.6
	Microsoft-Decision-1	25.9	47.2	47.2	47.2	47.2	47.2	46.7	46.7	46.7	44.0	43.5

Table 7: Test F1 of every frozen version (UMR: juːmætʃ; PARSEME: MWE-based). Blank: not tested (budget).

	Chinese				English			
	Haiku (fragm.)		Jev (closed)		Haiku (fragm.)		Jev (closed)	
Check	V_0	V_{10}	V_0	V_{10}	V_0	V_{10}	V_0	V_{10}
event concept	30.0	77.3	73.5	77.9	66.7	72.5	49.4	55.6
entity concept	36.3	73.8	64.4	69.7	69.5	76.3	68.1	72.5
granularity	82.5	85.8	92.2	92.9	84.9	88.1	90.3	90.7
event sense	58.6	84.7	84.4	86.1	63.2	71.9	56.8	60.9
aspect	60.5	65.7	51.7	63.1	52.0	61.3	55.2	57.5
polarity	52.6	61.1	56.4	49.0	36.8	36.8	32.4	37.8
refer (person/number)	20.5	23.2	42.6	49.0	35.3	49.9	35.8	37.7
core args of events	22.2	45.6	37.8	35.7	27.5	29.5	23.0	24.9
peripheral args of events	12.3	25.3	15.4	20.9	9.0	17.6	16.8	14.0
core role labels	60.2	75.1	74.8	68.6	65.4	60.9	69.7	71.8
name tokens	50.3	52.0	46.3	52.2	61.6	67.6	45.4	46.9
reentrancy	9.7	24.4	18.3	20.4	11.4	12.6	9.8	10.5
temporal	4.5	11.1	16.1	18.5	6.7	14.5	13.7	16.8
modal	48.7	54.6	55.2	56.5	28.6	31.7	28.1	27.2
coreference	3.0	16.4	4.3	12.7	11.9	22.5	9.3	11.3
juːmætʃ F1 (official)	20.9	33.2	33.9	37.6	28.8	32.6	29.8	31.8

Table 8: The numbers of Figure 5: the fine-grained UMR judge (F1 per check, %).

Lang.	System	correct	wrong cat.	partial	missed, no cand.	missed, rejected	spurious	F
Chinese	GPT-6.1-sol	25	3	4	27	0	21	50.0
	GPT-6-luna V_0	23	12	1	6	17	17	62.5
	GPT-6-luna V_{10}	33	4	1	6	15	5	72.5
	Haiku 5.5 V_0	26	7	4	3	19	16	59.5
	Haiku 5.5 V_{10}	22	3	3	5	26	8	53.2
	Jev 1.13 V_0	7	28	12	8	4	80	37.8
	Jev 1.13 V_{10}	22	10	1	11	15	9	63.4
	OpenAI Decisions V_0	15	12	10	8	14	59	35.1
	OpenAI Decisions V_{10}	18	6	0	11	24	5	54.5
	Microsoft-Decision-1 V_0	9	24	11	8	7	78	36.7
	Microsoft-Decision-1 V_{10}	16	13	5	8	17	34	46.0
English	GPT-6.1-sol	46	5	4	2	0	22	75.6
	GPT-6-luna V_0	31	7	8	6	5	21	62.3
	GPT-6-luna V_{10}	27	6	8	7	9	17	58.4
	Haiku 5.5 V_0	30	4	8	7	8	19	57.6
	Haiku 5.5 V_{10}	25	7	9	6	10	14	58.2
	Jev 1.13 V_0	19	6	20	10	2	112	22.5
	Jev 1.13 V_{10}	23	6	4	16	8	19	53.7
	OpenAI Decisions V_0	15	10	7	17	8	27	42.0
	OpenAI Decisions V_{10}	19	11	2	14	11	18	56.6
	Microsoft-Decision-1 V_0	17	13	15	11	1	123	25.9
	Microsoft-Decision-1 V_{10}	10	10	5	18	14	11	43.5

Table 9: The numbers of Figure 6: PARSEME error kinds on the test set (counts) and MWE-based F.

	UMR		PARSEME	
Decisions	zh	en	zh	en
GPT-6.1-sol + guideline	7.09	5.09	4.05	4.02
Haiku 5.5 (fragment)	1.88	0.83	0.03	0.02
GPT-6-luna (fragment)	–	–	0.01	0.01
Jev 1.13 (closed)	0.28	0.10	0.01	0.01
Microsoft-Decision-1 (closed)	0.19	0.07	0.01	0.01

Table 10: API cost (USD) of annotating the test set (95–100 sentences) with the seed procedure, uncached, as billed through OpenRouter. Reflection used Claude Sonnet 5.5 through a subscription; its list-price equivalent is $0.4–3.2 per ten-round chain.

System	Refl. docs (sent.)	T1: 0004–05 F/ Comp.	T2: 0005–07 F/ Comp.
Single call, Sonnet 5.5	0	32.7 / 27.7	29.4 / 24.9
R_0 (seed), Sonnet 5.5	0	45.3 / 44.0	–
R_{13}	1 (51)	50.8 / 49.4	–
R_{23}	2 (111)	52.6 / 51.2	–
R_{32}	3 (142)	52.6 / 51.1	55.2 / 52.1
R_{42}	4 (201)	n/a	55.6 / 52.2

Table 11: Earlier Chinese UMR runs with Sonnet 5.5 fragment decisions (AnCast++ sentence-graph F and comprehensive score, macro-averaged over documents). R_{xy}: the version after round y on training document x. "n/a": 0004 was a training document by then.

Doc	seed	iter 1	iter 2	iter 3	after
0001	40.3	40.3	59.6	66.9	70.9
0002	50.6	58.9	66.3	70.4	73.4
0003	49.7	57.5	69.5	–	–
0004	39.6	49.7	57.9	–	–

Table 12: The same runs: sentence F on each training document over the rounds of reflection on it. Training documents gain far more than test documents, as in the main text.